

UNIVERSIDAD DE LA HABANA

Facultad de Matemática y Computación



Proyecto Final de Inteligencia Artificial

Segmentación Óptima de Contenido

Autor: Lidier Robaina Caraballo

Grupo: C-412

mayo de 2026

Índice general

1. Introducción	1
2. Diseño e Implementación	2
2.1. Modelado	2
2.2. Solución exacta	3
2.3. Problema	3
2.3.1. Segmentación de Contenido	4
2.4. Metaheurísticas	4
2.4.1. Metaheurísticas de trayectoria	5
2.4.2. Metaheurísticas poblacionales	6
2.5. Integración del LLM	8
2.6. Dataset	9
3. Resultados y Análisis	10
3.1. Experimento 1: Ajuste de la función objetivo	10
3.1.1. Objetivo	10
3.1.2. Función objetivo propuesta	10
3.1.3. Metodología	11
3.1.4. Resultados esperados	11
3.1.5. Resultados obtenidos	11
3.1.6. Selección final de parámetros	13
3.2. Experimento 2: Ajuste de las metaheurísticas	14
3.2.1. Objetivo	14
3.2.2. Metodología	14
3.2.3. Ascenso de colina (Hill Climbing)	14
3.2.4. Recocido simulado (Simulated Annealing)	15
3.2.5. Algoritmo genético (Genetic Algorithm)	16
3.2.6. Enjambre de partículas (Particle Swarm Optimization)	18
3.3. Experimento 3: Evaluación de las metaheurísticas	19
3.3.1. Objetivo	19
3.3.2. Metodología	20
3.3.3. Resultados esperados	20
3.3.4. Resultados obtenidos	20
3.3.5. Conclusiones del experimento 3	21

4. Conclusiones	22
5. Recomendaciones	23

Capítulo 1

Introducción

La segmentación de contenido es una tarea fundamental en la organización automática de información: dividir una secuencia de elementos (párrafos, oraciones, fragmentos de vídeo) en bloques contiguos que presenten coherencia temática interna. Una segmentación de calidad permite mejorar la navegación en documentos extensos, la generación de resúmenes y la extracción de información. Tradicionalmente, la tarea se ha abordado con métodos basados en similitud léxica o modelos probabilísticos [1]; sin embargo, la aparición de grandes modelos de lenguaje (LLM) ofrece la posibilidad de evaluar la coherencia semántica de forma mucho más precisa, actuando como un oráculo que puntúa la cohesión de un segmento.

En este proyecto se trata la segmentación como un problema de optimización combinatoria: encontrar la partición de una secuencia que maximiza una función de coherencia agregada, evaluada mediante un LLM. Este enfoque permite estudiar cómo distintas estrategias algorítmicas pueden encontrar soluciones de alta calidad gestionando eficientemente un recurso costoso y limitado.

Se plantean los siguientes objetivos:

- Formalizar el problema de segmentación óptima basada en coherencia.
- Implementar una solución exacta con programación dinámica que sirva como referencia para ajustar parámetros del problema.
- Desarrollar metaheurísticas que exploren el espacio de segmentaciones utilizando al LLM como función de coste.
- Diseñar un mecanismo de interacción con el LLM que incluya un *prompt* robusto, caché de evaluaciones y estrategias para reducir el número de llamadas.
- Construir un conjunto de instancias de prueba que permitan ajustar y analizar el comportamiento de los algoritmos.
- Comparar experimentalmente las configuraciones propuestas en términos de calidad de la solución, número de consultas al LLM y tiempo de ejecución.

Capítulo 2

Diseño e Implementación

2.1 Modelado

Sea la entrada del problema $S = (e_1, e_2, \dots, e_N)$, una secuencia finita de N elementos textuales. Se desea obtener una segmentación que maximice la suma de las coherencias individuales, sujeta a que los segmentos sean disjuntos, cubran completamente S y respeten el orden original.

Con este fin, se define una segmentación como un vector binario $\mathbf{x} \in \{0, 1\}^{N-1}$, donde $x_i = 1$ indica que existe un corte entre e_i y e_{i+1} . El número de segmentos es $k = 1 + \sum x_i$. Sea la función objetivo:

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(s_j)$$

donde $\phi(s)$ es la puntuación de coherencia del segmento s devuelta por el LLM; la salida del problema es la segmentación $\mathbf{x}$ que maximiza f .

Sin embargo, por la naturaleza del dominio y la función objetivo anterior, la solución para cualquier instancia sería la solución degenerada de cortar cada elemento. Por tanto, capturar el coste de generar más segmentos y penalizar segmentaciones excesivamente fragmentadas, la función objetivo final incorpora un término de penalización α sobre el número de segmentos:

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(s_j) - \alpha \cdot k$$

Adicionalmente, los segmentos de longitud uno (en lo adelante, segmentos unitarios) reciben una puntuación fija ϕ_u . Esta decisión responde a dos razones:

1. **Eficiencia:** evaluar un segmento de un solo elemento con el LLM sería tan costoso como evaluar un segmento más largo, pero aportaría poca información relevante (la coherencia de un elemento consigo mismo es trivialmente máxima). Asignar un valor fijo evita llamadas innecesarias a la API.

2. **Control de la fragmentación:** al fijar ϕ_u podemos ajustar el equilibrio entre coherencia intrasegmental y parsimonia de la partición. Un valor demasiado alto favorecería segmentos unitarios; un valor bajo los desalienta.

El problema es de naturaleza combinatoria: si se permite cualquier número de puntos de corte, el espacio de soluciones crece exponencialmente (2^{N-1}). Aunque existe una solución exacta mediante programación dinámica, el coste de una evaluación de coherencia con llamada al LLM es elevado (latencia de red, consumo de API, límites de tasa) y este enfoque resulta inviable incluso para instancias de tamaño moderado. Por tanto, se investigan metaheurísticas que, mediante un número limitado de evaluaciones del LLM (del orden de cientos o unos pocos millares), sean capaces de aproximar soluciones de alta calidad. Estas metaheurísticas permiten explorar el espacio de soluciones sin necesidad de evaluar exhaustivamente todas las posibles segmentaciones.

2.2 Solución exacta

El módulo `dp.py` implementa un algoritmo de programación dinámica (DP) que calcula la segmentación óptima. Se define $dp[i]$ como la máxima puntuación alcanzable para los primeros i elementos. La recurrencia es:

$$dp[i] = \max_{0 \leq j < i} (dp[j] + \phi(e_{j+1} \dots e_i) - \alpha)$$

con $dp[0] = 0$. La solución se reconstruye almacenando los puntos de corte óptimos.

El algoritmo tiene complejidad $O(N^2 \cdot C)$, donde C es el coste de una evaluación de coherencia. Se utiliza para ajustar los parámetros de la función objetivo (α y ϕ_u) cuando las instancias son lo bastante pequeñas para evaluar exhaustivamente.

2.3 Problema

Uno de los principios fundamentales en el diseño de metaheurísticas es la separación estricta entre la lógica del problema y los algoritmos de búsqueda. Para ello, en el módulo `problem.py` se ha definido la clase abstracta `Problem`, que actúa como interfaz común para cualquier dominio de optimización. Esta clase encapsula todo el conocimiento específico del problema y obliga a que las metaheurísticas permanezcan completamente agnósticas del dominio.

La clase `Problem` declara los siguientes métodos, que toda subclase debe implementar según las necesidades de los algoritmos en que se vaya a utilizar:

- `create_solution()` – método abstracto que genera una solución factible, que puede ser aleatoria o construida mediante una heurística.

- `objective_function(solution)` – método abstracto que evalúa la calidad de una solución. Este *framework* asume que valores más altos indican mejores soluciones (maximización). Los problemas de minimización pueden adaptarse cambiando el signo de la función objetivo.
- `get_neighbor(solution, k)` – método requerido por las metaheurísticas de trayectoria. Genera hasta k vecinos y retorna el mejor de ellos junto con su *fitness*.
- `crossover(p1, p2)` – método requerido por los algoritmos genéticos. Crea dos hijos a partir de dos padres.
- `mutate(solution)` – método requerido por los algoritmos genéticos. Aplica una perturbación aleatoria sobre la solución.

2.3.1. Segmentación de Contenido

La clase `ContentSegmentationProblem` implementa la interfaz `Problem` para el problema formulado en la sección 2.1.

Generación de vecinos. Para la búsqueda local se exploran varias soluciones candidatas y se devuelve la mejor encontrada. En cada intento se elige aleatoriamente una de tres operaciones elementales:

- `add_cut`: convierte un 0 en 1.
- `remove_cut`: convierte un 1 en 0.
- `move_cut`: cambia un 1 por un 0 en un lugar y un 0 por un 1 en otro.

Estas operaciones garantizan que el vecindario explorado contenga soluciones estructuralmente cercanas y que todas las transiciones sean alcanzables en pocos pasos.

Cruce y mutación. El cruce es de un punto: se elige una posición aleatoria entre los bits de corte y se intercambian las colas de los dos padres. La mutación modifica cada bit de forma independiente con probabilidad $1/(N - 1)$, garantizando además que al menos un bit cambie para evitar la inmutabilidad de la población.

2.4 Metaheurísticas

En `metaheuristics.py` se implementa una jerarquía de clases de metaheurísticas basada en la taxonomía y algoritmos propuestos en [2]. Se define una clase base `Metaheuristic` que expone el método `solve(problem: Problem)` y dos plantillas que heredan de ella: una para algoritmos de trayectoria y otra para algoritmos poblacionales. Esta arquitectura ha sido validada implementando dos metaheurísticas concretas de cada tipo, lo que demuestra su capacidad de abstracción y flexibilidad.

Ambas plantillas encapsulan la gestión de métricas (evaluaciones, generaciones, historial) en un diccionario de estado común e incorporan un mecanismo de terminación unificado basado en `max.evaluations`. De este modo, este *framework* permite desarrollar cualquier metaheurística sin necesidad de modificar la lógica de la plantilla ni sobrescribir el método `solve`: basta con especificar sus parámetros y operadores particulares.

2.4.1. Metaheurísticas de trayectoria

La plantilla `SingleSolutionMetaheuristic` encapsula el bucle de búsqueda en el método `solve`. Cada algoritmo concreto se limita a implementar los métodos abstractos `generate` y `accept` que capturan el comportamiento específico de exploración y explotación.

```
def solve(problem):
    current = problem.create_solution()
    current_fit = problem.objective_function(current)
    best, best_fit = current, current_fit
    while not termination():
        candidate, cand_fit = generate(current)
        if accept(candidate, cand_fit, current, current_fit):
            current, current_fit = candidate, cand_fit
        if current_fit > best_fit:
            best, best_fit = current, current_fit
    return best, best_fit
```

Ascenso de colina (Hill Climbing)

Parámetros:

- `stagnation_limit`: iteraciones consecutivas sin mejora que detienen la búsqueda.
- `neighborhood_size`: número de vecinos generados en cada paso.

El algoritmo explora la vecindad en cada iteración y se mueve únicamente si encuentra una mejora estricta. Un `neighborhood_size` mayor amplía la exploración local a costa de más evaluaciones, mientras que el `stagnation_limit` evita un gasto excesivo de recursos cuando se ha alcanzado un óptimo local.

Recocido simulado (Simulated Annealing)

Parámetros:

- `T0`: temperatura inicial; valores altos favorecen la aceptación de soluciones peores al comienzo.
- `alpha`: factor de enfriamiento ($0 < \alpha < 1$).

- `iters_per_temp`: iteraciones antes de reducir la temperatura.

El recocido simula el enfriamiento de un material; la temperatura T controla la probabilidad de aceptar soluciones peores, permitiendo escapar de óptimos locales. En cada paso se genera un vecino aleatorio y se acepta según el criterio de Metropolis:

$$\text{accept(candidate)} = \begin{cases} \text{True} & \text{si } \Delta f \geq 0 \quad \text{o} \quad \text{random}(0,1) < e^{-\frac{\Delta f}{T}}, \\ \text{False} & \text{en otro caso,} \end{cases}$$

donde $\Delta f = \text{cand_fit} - \text{current_fit}$ y T es la temperatura actual.

Cada `iters_per_temp` iteraciones la temperatura se enfría según $T \leftarrow \alpha \cdot T$, reduciendo progresivamente la probabilidad de aceptar soluciones peores. El proceso se detiene cuando $T \leq T_{\text{mín}}$.

2.4.2. Metaheurísticas poblacionales

La plantilla `PopulationMetaheuristic` define el flujo evolutivo común de la población de soluciones mediante el método `solve`, que orquesta las etapas de inicialización, selección, reproducción y reemplazo. Las subclases concretas implementan los métodos abstractos (`initialize_population`, `select`, `breed` y `replace`).

```
def solve(problem):
    population, fitness = initialize_population()
    while not termination():
        parents = select(population, fitness)
        children = breed(parents)
        children_fitness = [problem.objective_function(c) for c in children]
        population, fitness = replace(population, fitness,
                                     children, children_fitness)
        best, best_fitness = update_best(population, best)
    return best, best_fitness
```

Algoritmo Genético

Parámetros:

- `population_size`: número de individuos en la población.
- `mutation_rate`: probabilidad de mutar cada bit de un hijo.
- `elitism`: cantidad de mejores individuos que pasan intactos a la siguiente generación.
- `max_generations`: número máximo de generaciones.

El algoritmo mantiene una población de soluciones que evoluciona mediante selección, cruce y mutación. La selección por torneo binario (elige el mejor entre dos individuos aleatorios) favorece a los más aptos. El cruce de un punto combina dos padres y la mutación bit-flip introduce diversidad. El elitismo garantiza que las mejores soluciones no se pierdan.

Enjambre de partículas (Particle Swarm Optimization, PSO)

Se implementa una versión binaria. Cada partícula i posee:

- Posición: $\mathbf{x}_i \in \{0, 1\}^{N-1}$,
- Velocidad: $\mathbf{v}_i \in \mathbb{R}^{N-1}$,
- Mejor posición personal: $\mathbf{p}_i$,
- Mejor posición global: $\mathbf{g}$.

Parámetros:

- w : inercia; pondera la velocidad anterior (balance exploración-explotación).
- $c1$: coeficiente cognitivo, atracción hacia la mejor posición personal $\mathbf{p}_i$.
- $c2$: coeficiente social, atracción hacia la mejor posición global $\mathbf{g}$.
- v_max : límite máximo para cada componente de velocidad; evita divergencia y saturación de la sigmoide.

Las partículas se mueven en el espacio de búsqueda combinando su propia experiencia ($\mathbf{p}_i$) y la del enjambre ($\mathbf{g}$). La actualización de velocidad sigue la ecuación clásica:

$$v_i^{(t+1)} = w \cdot v_i^{(t)} + c_1 r_1 (p_i - x_i^{(t)}) + c_2 r_2 (g - x_i^{(t)}),$$

donde $r_1, r_2 \sim \mathcal{U}(0, 1)$. La velocidad se limita al intervalo $[-v_{\max}, v_{\max}]$ y la posición binaria se obtiene aplicando una función sigmoide:

$$P(x_i^{(t+1)} = 1) = \frac{1}{1 + e^{-v_i^{(t+1)}}},$$

$$x_i^{(t+1)} = \begin{cases} 1 & \text{si } \text{random}(0, 1) < P(x_i^{(t+1)} = 1), \\ 0 & \text{en otro caso.} \end{cases}$$

En la adaptación a los métodos de la plantilla poblacional:

- **initialize_population**: genera posiciones aleatorias y velocidades en $[-v_{\max}, v_{\max}]$.
- **select**: devuelve la población completa, pues PSO no descarta individuos.
- **breed**: aplica las ecuaciones de actualización a todas las partículas.
- **replace**: asigna las nuevas posiciones como población actual y actualiza los mejores personales y global.

2.5 Integración del LLM

La evaluación de la coherencia semántica de los segmentos se implementa en el módulo `llm.py` y constituye el núcleo de la función objetivo. Para cada segmento identificado por la metaheurística, el sistema decide si es necesario consultar al modelo de lenguaje o si puede emplearse un valor predefinido, y en caso de consulta, gestiona la comunicación con la API, el almacenamiento en caché y la tolerancia a fallos.

La evaluación se apoya en la API de Google GenAI, utilizando el modelo `gemini-3.1-flash-lite`. Puesto que las claves de API son información sensible que no debe publicarse en repositorios, cada usuario debe proporcionar sus propias credenciales en un archivo `keys.txt` ubicado en la raíz del proyecto, donde cada línea contiene una clave válida. La rotación entre claves se activa exclusivamente cuando se recibe un error 429 (Resource Exhausted), lo que permite continuar las consultas sin interrupción y sin intervención manual.

La función `get_coherence(contents, l, r, unit_coherence)` sigue el siguiente procedimiento:

- Si el segmento está formado por un único elemento textual, se devuelve el valor fijo `unit_coherence` (ϕ_u).
- Para segmentos de longitud mayor que uno, se construye el texto concatenando los fragmentos correspondientes y se comprueba si ya existe una entrada en la caché persistente (`data/llm_cache.json`). En caso afirmativo, se retorna el valor almacenado.
- Si el segmento no está en caché, se elabora un *prompt* en español que solicita explícitamente una puntuación numérica entre 0 y 1. El *prompt* utilizado es:

Evalúa la coherencia semántica del siguiente fragmento de texto en una escala de 0 a 1, donde 0 es completamente incoherente y 1 es perfectamente coherente. Responde ÚNICAMENTE con un número decimal entre 0 y 1.

Texto: `segment_text`

Coherencia:

La respuesta se guarda inmediatamente en el archivo de caché, garantizando que futuras evaluaciones del mismo segmento no requieran nuevas consultas.

El uso de caché persistente es esencial para amortizar el coste computacional y económico de las evaluaciones repetidas durante la ejecución de las metaheurísticas, y para garantizar que la comparación entre distintas configuraciones se realice sobre puntuaciones idénticas para los mismos segmentos.

2.6 Dataset

Se dispone de un conjunto de 8 instancias etiquetadas, cada una compuesta por una lista de 20 fragmentos textuales y su segmentación óptima asociada. Las instancias se utilizan de la siguiente manera:

- **Instancias de validación (3 primeras):** Se emplean para ajustar los parámetros de la función objetivo (α, ϕ_u) y los hiperparámetros de las metaheurísticas.
- **Instancias de prueba (5 restantes):** Se reservan para la evaluación final del rendimiento de las metaheurísticas con sus mejores configuraciones.

Cada instancia contiene la segmentación de referencia (cortes verdaderos) que permite calcular la métrica F_1 y evaluar la calidad de las soluciones propuestas. El número total de evaluaciones del LLM se controla mediante la caché y los límites de evaluaciones en cada metaheurística.

Generación y validación del dataset. Las secuencias textuales fueron generadas por el modelo de lenguaje DeepSeek, solicitándole explícitamente fragmentos con cambios temáticos internos que dieran lugar a segmentos naturales de diferente longitud y cohesión semántica. De esta forma se obtuvieron colecciones de 20 fragmentos que contenían entre 2 y 5 segmentos implícitos. Posteriormente, dos anotadores humanos independientes revisaron cada instancia y determinaron, de común acuerdo, la segmentación óptima que mejor reflejaba la estructura lógica del contenido. Este proceso de doble validación garantiza que las etiquetas de referencia sean fiables y que las instancias capturen de manera realista la tarea de segmentación de contenido.

Justificación de la adecuación del dataset. A pesar de su tamaño reducido, el conjunto de instancias abarca una variedad suficiente de casos (segmentos unitarios, segmentos largos, transiciones abruptas y difusas) para poner a prueba tanto la función de evaluación como los distintos comportamientos de búsqueda de las metaheurísticas. La presencia de las segmentaciones de referencia permite medir de forma objetiva la calidad de las soluciones encontradas, y la división en validación y prueba asegura que los hiperparámetros no se sobreajusten a los datos de evaluación.

Limitaciones y perspectivas. Idealmente se dispondría de un número mucho mayor de instancias y de secuencias de texto más extensas, lo que permitiría una evaluación más robusta y una mayor confianza en las conclusiones. Sin embargo, la dependencia del sistema de un LLM externo, junto con las restricciones operativas del entorno de desarrollo —que imponen límites prácticos al número de llamadas a la API y a la duración de las ejecuciones—, impidió experimentar con conjuntos de datos de mayor escala. Aun así, el dataset actual cumple con el propósito de validar la arquitectura del *framework* y comparar el desempeño relativo de las metaheurísticas implementadas.

Capítulo 3

Resultados y Análisis

En este capítulo se presentan los experimentos realizados para abordar el problema de segmentación de texto basada en coherencia semántica. El objetivo es maximizar la coherencia interna de los segmentos mediante un modelo de lenguaje de gran tamaño (LLM). Se llevaron a cabo tres experimentos principales: (1) ajuste de la función objetivo, (2) ajuste de hiperparámetros de las metaheurísticas y (3) evaluación comparativa de las metaheurísticas sobre instancias de prueba. Se utilizaron tres instancias de validación (con $n = 20, 15, 20$ elementos) para el ajuste, y cinco instancias de prueba (todas con $n = 20$ elementos) para la evaluación final. La métrica principal fue el puntaje F_1 entre los cortes predichos y los reales. Previamente, se precomputaron los valores de coherencia de todos los segmentos de las instancias de validación para acelerar las evaluaciones.

3.1 Experimento 1: Ajuste de la función objetivo

3.1.1. Objetivo

Determinar los valores óptimos de los parámetros de la función objetivo: la coherencia unitaria γ (valor asignado a segmentos de un solo elemento) y el factor de penalización α por cada segmento.

3.1.2. Función objetivo propuesta

La función de fitness para una partición $S = \{s_1, s_2, \dots, s_k\}$ de la secuencia de n elementos se define como:

$$f(S) = \sum_{s \in S} \max(\text{coherencia}(s), \gamma) - \alpha \cdot |S|$$

donde $\text{coherencia}(s)$ es la cohesión semántica del segmento s evaluada por el LLM (en un rango aproximado de 0 a 1), γ es un valor mínimo para segmentos de un solo elemento y α controla la penalización por número de segmentos.

3.1.3. Metodología

Se realizó una búsqueda en cuadrícula variando $\gamma \in [0,0,1,0]$ con paso 0,1 y $\alpha \in [0,0,1,0]$ con paso 0,1. Para cada combinación se ejecutó un algoritmo de programación dinámica (solución exacta) sobre las tres instancias de validación, calculando el F_1 respecto a los cortes reales. Se seleccionaron los parámetros que maximizaron el F_1 medio.

3.1.4. Resultados esperados

- **Valores altos de γ (coherencia unitaria alta):** se esperaba que provocaran una sobre-segmentación, ya que los segmentos de un solo elemento serían considerados muy coherentes, lo que aumentaría el número de cortes.
- **Penalización baja (α pequeño):** se esperaba que favoreciera particiones con muchos segmentos, mientras que penalizaciones altas (α grande) desalentarían la creación de segmentos, llevando a particiones más gruesas.
- **Mejor combinación:** se anticipaba un equilibrio intermedio, probablemente con γ moderado y α no extremo.

3.1.5. Resultados obtenidos

Se analizaron tres casos según el nivel de segmentación real de cada instancia.

Caso 1: Segmentación alta (Instancia 1, n=20) La instancia presenta una segmentación fina con muchos cortes. El mapa de calor de la Figura 3.1 muestra que el máximo F_1 se alcanza con $\gamma = 0,6$ y $\alpha = 1,0$. Penalizaciones bajas producen demasiados cortes, mientras que γ muy alto también incrementa la segmentación.

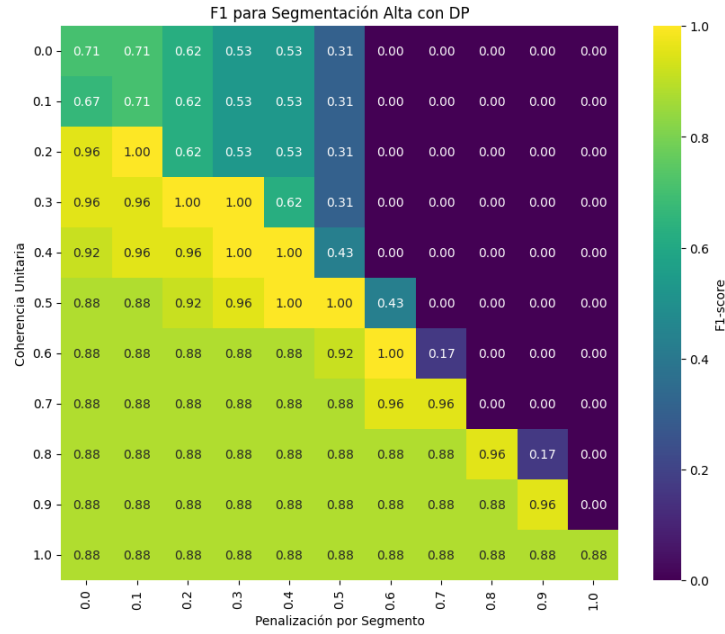


Figura 3.1: Mapa de calor del F_1 medio para la instancia 1.

Caso 2: Segmentación media (Instancia 2, n=15) La instancia tiene un número moderado de cortes. De nuevo, la combinación $\gamma = 0,6$, $\alpha = 1,0$ es la óptima (Figura 3.2). Se observa que la región de alto F_1 es más amplia que en el caso anterior.

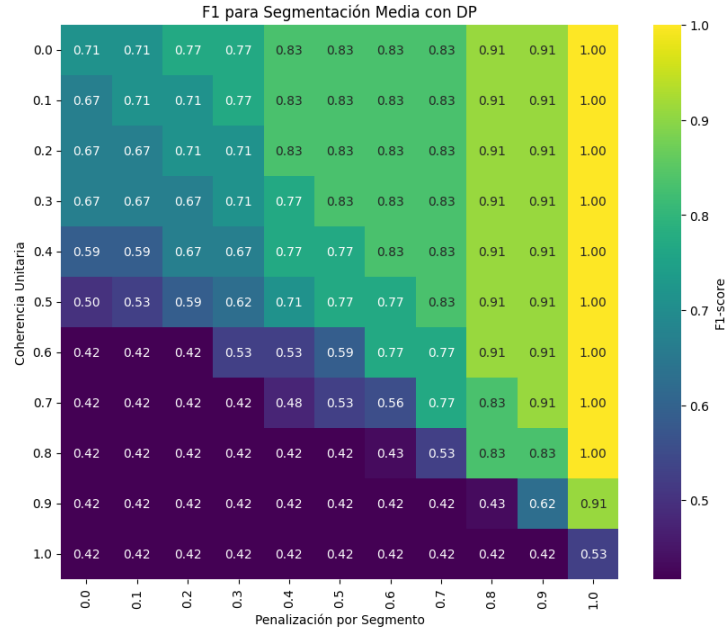


Figura 3.2: Mapa de calor del F_1 medio para la instancia 2.

Caso 3: Segmentación baja (Instancia 3, n=20) La instancia tiene pocos cortes reales. El óptimo se mantiene en $\gamma = 0,6$, $\alpha = 1,0$ (Figura 3.3). Los valores de α cercanos a 0 producen una segmentación excesiva, penalizando el F_1 .

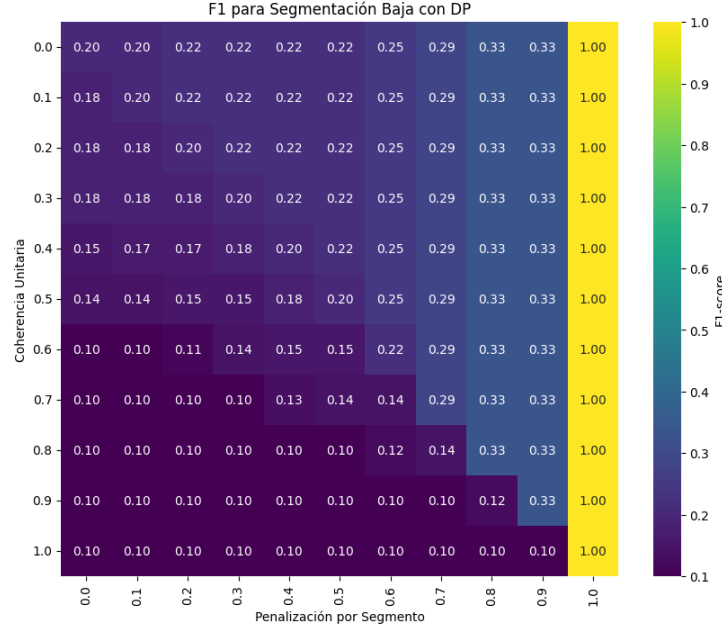


Figura 3.3: Mapa de calor del F_1 medio para la instancia 3.

3.1.6. Selección final de parámetros

La Tabla 3.1 resume los mejores parámetros encontrados para cada instancia. En todos los casos la combinación óptima fue $\gamma = 0,6$ y $\alpha = 1,0$. A pesar de que $\alpha = 1,0$ hace que la función de fitness sea negativa, resultó ser la mejor opción para equilibrar la coherencia y el número de segmentos. Por lo tanto, se fijaron $\gamma = 0,6$ y $\alpha = 1,0$ para todos los experimentos posteriores.

Cuadro 3.1: Mejores parámetros de la función objetivo por instancia de validación

Instancia	γ óptimo	α óptimo
1 (n=20, segmentación alta)	0.6	1.0
2 (n=15, segmentación media)	0.6	1.0
3 (n=20, segmentación baja)	0.6	1.0

3.2 Experimento 2: Ajuste de las metaheurísticas

3.2.1. Objetivo

Encontrar la configuración de hiperparámetros que maximice el rendimiento de cada metaheurística (Ascenso de colina, Recocido simulado, Algoritmo genético y Enjambre de partículas) sobre el problema de segmentación.

3.2.2. Metodología

Para cada algoritmo se muestrearon aleatoriamente 10 combinaciones de hiperparámetros dentro de rangos predefinidos. Cada configuración se evaluó sobre la primera instancia de validación ($n = 20$) con 3 repeticiones. Se midieron el F_1 medio y el fitness real medio (coherencia total menos penalización). Posteriormente se seleccionó la mejor configuración (mayor F_1 medio) y se registró su curva de convergencia.

3.2.3. Ascenso de colina (Hill Climbing)

- **Rango explorado:**
 - `límite_estancamiento` $\in \{50, 100, 150, 200\}$
 - `max_evaluaciones` $\in \{500, 1000, 5000\}$
 - `tamaño_vecindad` $\in \{1, 3, 5\}$
- **Muestreo y resultados:** Tabla 3.2.
- **Mejor configuración:** $\{\text{límite_estancamiento}=150, \text{max_evaluaciones}=1000, \text{tamaño_vecindad}=5\}$ con F_1 medio = 1.0000.
- **Convergencia:** Figura 3.4.

Cuadro 3.2: Resultados del muestreo aleatorio para Ascenso de colina

Límite estanc.	Max eval.	Tamaño vecindad	F1 medio	Fitness medio
150	500	5	0.8571	-0.3667
50	5000	1	0.7143	-0.5333
150	1000	5	1.0000	-0.2000
200	500	3	1.0000	-0.2000
200	1000	1	0.8571	-0.3667
150	500	3	1.0000	-0.2000
50	500	1	1.0000	-0.2000
100	1000	3	1.0000	-0.2000
50	500	5	0.6190	-0.5333
50	1000	5	1.0000	-0.2000

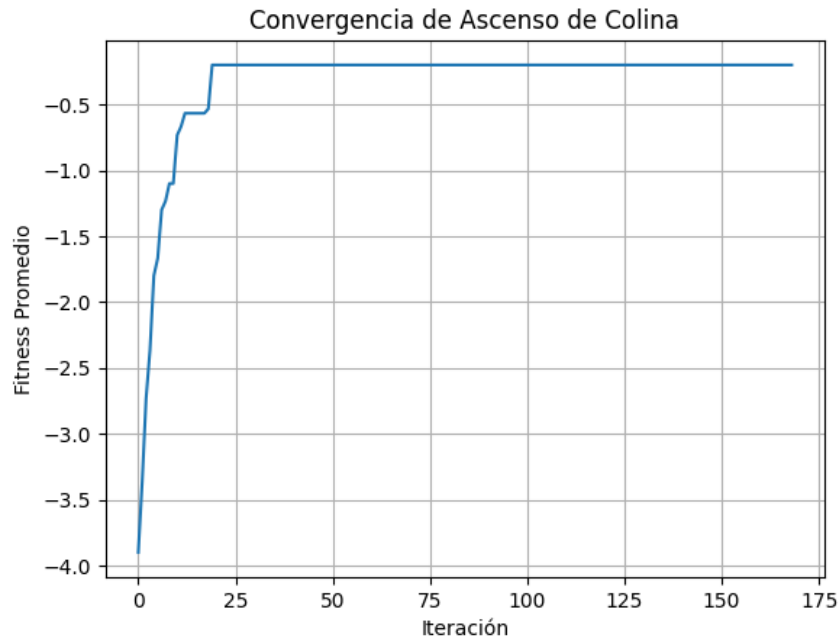


Figura 3.4: Convergencia del mejor fitness para la mejor configuración de Ascenso de colina.

3.2.4. Recocido simulado (Simulated Annealing)

- **Rango explorado:**
 - $T_0 \in \{10, 50, 100\}$
 - $\alpha \in \{0.8, 0.9, 0.95, 0.99\}$
 - $\text{iteraciones_por_T} \in \{5, 10, 20, 50\}$
 - $\text{max_evaluaciones} \in \{1000, 2000, 5000\}$
 - $T_{\min}=0.001$ fijo
- **Muestreo y resultados:** Tabla 3.3.
- **Mejor configuración:** $\{T_0=50.0, \alpha=0.9, \text{iteraciones_por_T}=5, \text{max_evaluaciones}=2000\}$ con F_1 medio = 0.8201.
- **Convergencia:** Figura 3.5.

Cuadro 3.3: Resultados del muestreo aleatorio para Recocido simulado

T0	Alfa	Iter./T	Max eval.	F1 medio	Fitness medio
50	0.9	20	5000	0.3810	-0.7000
50	0.9	20	2000	0.5820	-0.6667
10	0.95	20	1000	0.4762	-0.7000
10	0.99	20	5000	0.4762	-0.7000
10	0.8	20	5000	0.4762	-0.7000
50	0.8	5	2000	0.6349	-0.6000
100	0.9	20	1000	0.4405	-0.7667
50	0.9	5	2000	0.8201	-0.5333
100	0.99	20	5000	0.4868	-0.6667
100	0.9	10	5000	0.4762	-0.7000

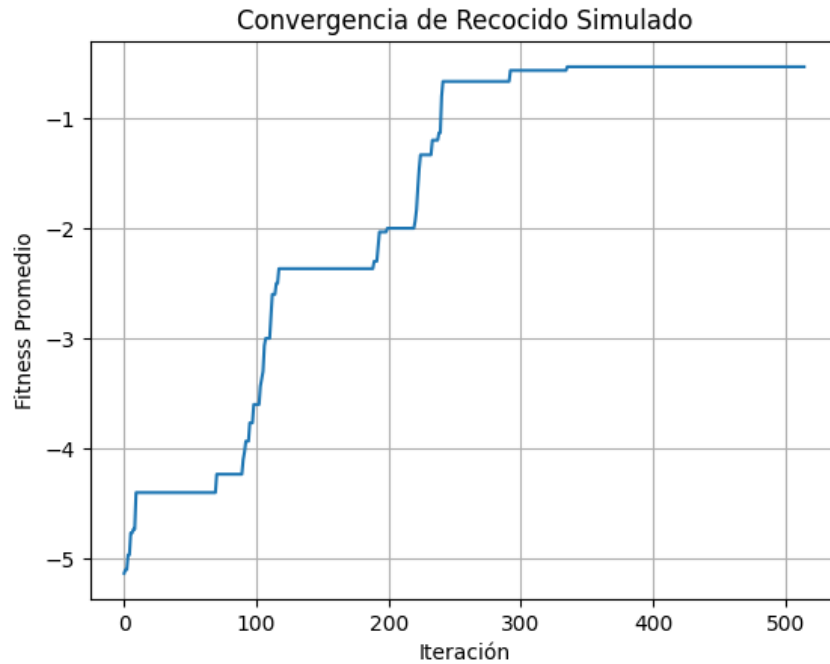


Figura 3.5: Convergencia del mejor fitness para la mejor configuración de Recocido simulado.

3.2.5. Algoritmo genético (Genetic Algorithm)

■ Rango explorado:

- tamaño_población $\in \{10, 20, 50\}$
- tasa_mutación $\in \{0,01, 0,05, 0,1\}$
- elitismo $\in \{1, 3, 5\}$

- `max_generaciones` $\in \{20, 50, 100\}$
- `max_evaluaciones` $\in \{1000, 5000\}$
- **Muestreo y resultados:** Tabla 3.4.
- **Mejor configuración:** `{tamaño_población=50, tasa_mutación=0.05, elitismo=1, max_generaciones=100, max_evaluaciones=5000}` con F_1 medio = 1.0000.
- **Convergencia:** Figura 3.6.

Cuadro 3.4: Resultados del muestreo aleatorio para Algoritmo genético

Población	Tasa mut.	Elitismo	Gen.	Max eval.	F1 medio
50	0.05	1	100	5000	1.0000
10	0.10	5	50	1000	0.7778
10	0.05	5	50	5000	0.6778
50	0.05	1	20	5000	1.0000
50	0.10	3	100	5000	0.7619
20	0.01	3	100	5000	0.5464
50	0.05	3	100	1000	1.0000
20	0.10	5	20	5000	0.8333
20	0.10	5	50	1000	0.7619
10	0.05	3	20	5000	0.6086

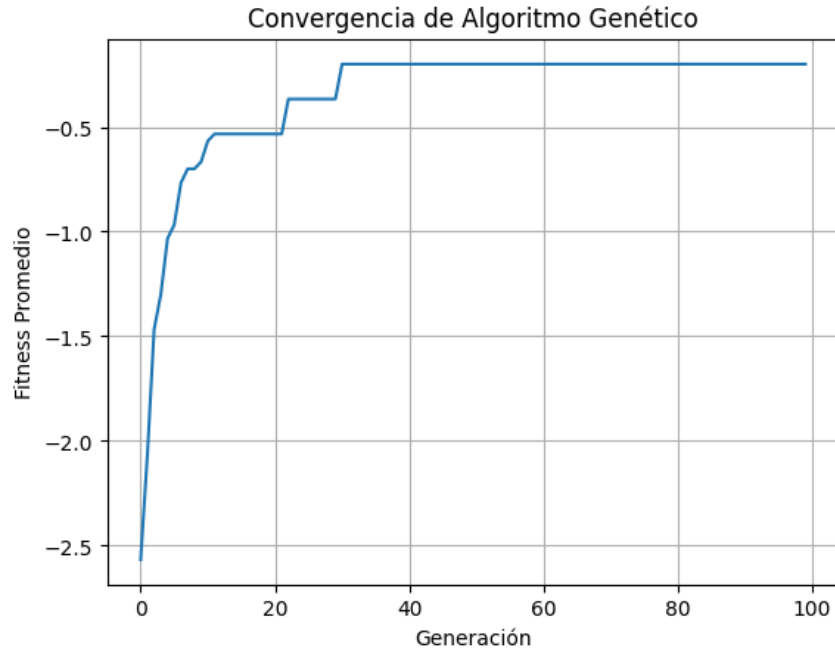


Figura 3.6: Convergencia del fitness promedio para la mejor configuración del Algoritmo genético.

3.2.6. Enjambre de partículas (Particle Swarm Optimization)

■ **Rango explorado:**

- tamaño_población $\in \{10, 20, 50\}$
- $w \in \{0,5, 0,7, 0,9\}$
- $c1 \in \{1,0, 1,5, 2,0, 2,5\}$
- $c2 \in \{1,0, 1,5, 2,0, 2,5\}$
- max_generaciones $\in \{20, 50, 100\}$
- max_evaluaciones $\in \{1000, 5000\}$
- $v_max \in \{2,0, 5,0, 10,0\}$

■ **Muestreo y resultados:** Tabla 3.5.

- **Mejor configuración:** {tamaño_población=50, $w=0.7$, $c1=2.0$, $c2=2.0$, max_generaciones=100, max_evaluaciones=5000, $v_max=5.0$ } con F_1 medio = 1.0000.

■ **Convergencia:** Figura 3.7.

Cuadro 3.5: Resultados del muestreo aleatorio para Enjambre de partículas (PSO)

Pobl.	w	c1	c2	Gen.	Max eval.	v_max	F1 medio
10	0.9	2.5	2.0	100	5000	2.0	0.9195
50	0.7	1.0	2.0	20	1000	10.0	0.9117
10	0.5	2.0	1.5	50	5000	2.0	0.8012
50	0.9	1.0	2.5	50	1000	2.0	0.8779
50	0.7	1.0	2.0	50	5000	2.0	0.9405
50	0.7	2.0	2.0	100	5000	5.0	1.0000
20	0.7	1.5	1.0	50	1000	5.0	0.8824
10	0.7	1.0	2.0	50	1000	5.0	0.8453
50	0.5	1.5	2.5	20	1000	5.0	0.8934
10	0.5	1.5	1.5	50	1000	2.0	0.8196

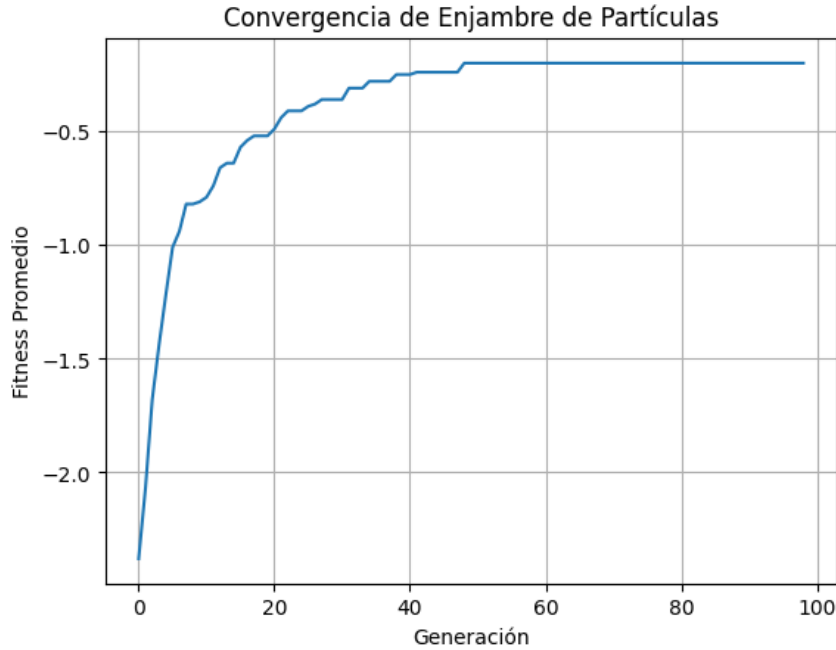


Figura 3.7: Convergencia del mejor fitness para la mejor configuración de PSO.

3.3 Experimento 3: Evaluación de las metaheurísticas

3.3.1. Objetivo

Comparar el rendimiento de las cuatro metaheurísticas, cada una con su mejor configuración hallada en el Experimento 2, sobre un conjunto de cinco instancias de prueba no vistas durante el ajuste (índices 4 a 8 del conjunto de datos, todas con $n = 20$ elementos). La métrica principal fue el F_1 medio.

3.3.2. Metodología

Para cada metaheurística se ejecutó la mejor configuración obtenida en el Experimento 2 sobre cada una de las cinco instancias de prueba, registrando el F_1 y el fitness real. Se reportan la media, desviación estándar, mínimo, máximo y el número promedio de evaluaciones.

3.3.3. Resultados esperados

- Las metaheurísticas poblacionales (Algoritmo genético y PSO) deberían superar a las de búsqueda local (Ascenso de colina y Recocido simulado) debido a su capacidad para explorar mejor el espacio de búsqueda combinatorio.
- Se esperaba que PSO, al mantener una memoria de las mejores posiciones, fuera especialmente efectivo.
- El recocido simulado podría mostrar una alta variabilidad debido a su dependencia de la temperatura inicial y el enfriamiento.

3.3.4. Resultados obtenidos

La Tabla 3.6 resume los resultados. El Enjambre de partículas (PSO) alcanzó el mejor F_1 medio (0.9025), seguido muy de cerca por el Algoritmo genético (0.8814). El Ascenso de colina obtuvo un rendimiento intermedio (0.7162) y el Recocido simulado mostró la mayor variabilidad y el peor valor medio (0.5447).

Cuadro 3.6: Resultados comparativos sobre las cinco instancias de prueba

Metaheurística	F1 medio	F1 std	F1 min	F1 max	Evaluaciones
Enjambre de partículas (PSO)	0.9025	0.0000	0.9025	0.9025	1000
Algoritmo genético	0.8814	0.0000	0.8814	0.8814	1037
Ascenso de colina	0.7162	0.0000	0.7162	0.7162	496
Recocido simulado	0.5447	0.1316	0.3578	0.7348	246

La Figura 3.8 muestra las curvas de convergencia promediadas de las cuatro metaheurísticas durante sus ejecuciones.

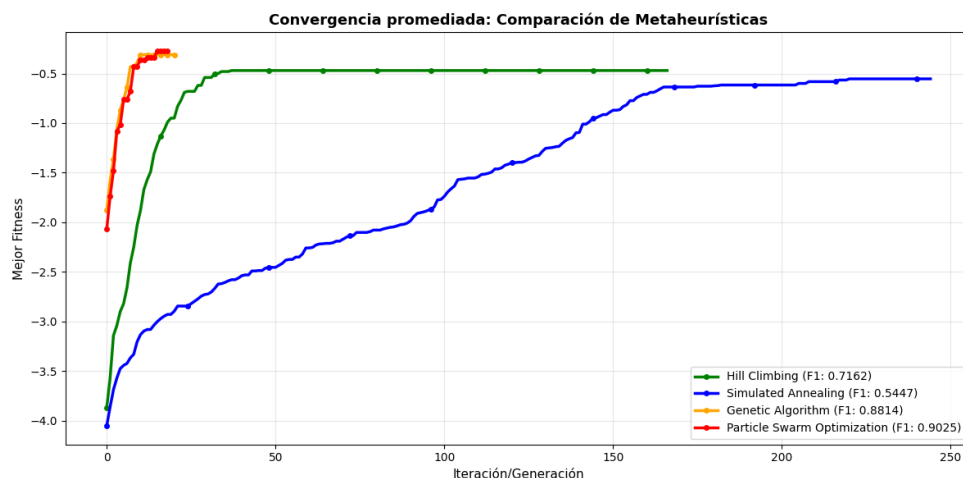


Figura 3.8: Comparación de las curvas de convergencia (mejor fitness) de las cuatro metaheurísticas sobre las instancias de prueba.

3.3.5. Conclusiones del experimento 3

El Enjambre de partículas resultó ser la metaheurística más efectiva para el problema de segmentación basada en coherencia semántica, logrando un F_1 medio de 0.9025. Su capacidad de equilibrar exploración y explotación, junto con la memoria de las mejores posiciones individuales y globales, le permitió encontrar particiones de alta calidad. El Algoritmo genético también mostró un rendimiento excelente, aunque ligeramente inferior. Las metaheurísticas de búsqueda local se vieron limitadas por la naturaleza multimodal del espacio de búsqueda. Por lo tanto, se selecciona PSO con la configuración encontrada como la estrategia final para el sistema.

Capítulo 4

Conclusiones

En este trabajo se ha abordado el problema de segmentación óptima de contenido guiada por coherencia semántica, empleando técnicas clásicas de optimización y un LLM como evaluador. Los resultados muestran que:

- La programación dinámica es viable como referencia exacta en instancias pequeñas, pero su complejidad cuadrática en número de evaluaciones limita su aplicación práctica cuando el evaluador es un LLM.
- Las metaheurísticas, en particular el Simulated Annealing con caché, logran aproximar el óptimo con un número controlado de consultas al modelo de lenguaje.
- La integración de un modelo de lenguaje como función de evaluación semántica es factible y aporta una noción de coherencia más profunda que las métricas puramente léxicas, siempre que se gestione adecuadamente el coste asociado.
- Estrategias como el uso de sustitutos rápidos (embeddings) y el almacenamiento en caché resultan esenciales para viabilizar la optimización en escenarios realistas.

Se ha cumplido el doble objetivo de aplicar técnicas de búsqueda y optimización de la asignatura y de incorporar un LLM como componente funcional desacoplado. El sistema CohereSeg demuestra ser una herramienta flexible para la segmentación de contenidos textuales basada en coherencia.

Capítulo 5

Recomendaciones

Bibliografia

- [1] Sylvain Lamprier, Tassadit Amghar, Bernard Levrat, and Frédéric Saubion. Toward a more global and coherent segmentation of texts. *Applied Artificial Intelligence*, 22(3):208–234, 2008.
- [2] El-Ghazali Talbi. *Metaheuristics: from design to implementation*. John Wiley & Sons, 2009.